

Bayes Optimal Classifier Gibbs

Student Placement: MDL, Bayes Optimal, and Gibbs Sampling

This notebook demonstrates machine learning concepts using a student dataset.

[1]
✓ Os

```
import pandas as pd
import numpy as np

data = {
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 2, 4, 6, 7, 8, 3, 5, 6, 9, 1, 4, 7, 8],
    "Attendance": [45, 50, 55, 65, 70, 80, 90, 40, 60, 75, 85, 92, 48, 68, 78, 95, 35, 58, 82, 88],
    "Project_Done": [0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1],
    "Placed": [0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1]
}

df = pd.DataFrame(data)
display(df.head())
print("\nStudent Placement Dataset Full:")
print(df)
```

✓

	Study_Hours	Attendance	Project_Done	Placed
0	2	45	0	0
1	3	50	0	0
2	4	55	0	0
3	5	65	1	0
4	6	70	1	1

Student Placement Dataset Full:

✓

```
Student Placement Dataset Full:
  Study_Hours  Attendance  Project_Done  Placed
0           2          45             0        0
1           3          50             0        0
2           4          55             0        0
3           5          65             1        0
4           6          70             1        1
5           7          80             1        1
6           8          90             1        1
7           2          40             0        0
8           4          60             0        0
9           6          75             1        1
10          7          85             1        1
11          8          92             1        1
12          3          48             0        0
13          5          68             1        1
14          6          78             1        1
15          9          95             1        1
16          1          35             0        0
17          4          58             1        0
18          7          82             1        1
19          8          88             1        1
```

1. Minimum Description Length (MDL)

We compare different rules (hypotheses) by balancing their complexity and error rates.

```
actual = df["Placed"].to_numpy()

# Candidate Rule 1
rule1_prediction = (df["Study_Hours"] >= 6).astype(int).to_numpy()
# Candidate Rule 2
rule2_prediction = (df["Attendance"] >= 70).astype(int).to_numpy()
# Candidate Rule 3
rule3_prediction = ((df["Study_Hours"] >= 5) & (df["Attendance"] >= 65) & (df["Project_Done"] == 1)).as

# Calculate MDL components
rule1_errors, rule2_errors, rule3_errors = [np.sum(p != actual) for p in [rule1_prediction, rule2_predi
rule1_complexity, rule2_complexity, rule3_complexity = 3, 3, 7

mdl_results = pd.DataFrame({
    "Rule": ["Hours >= 6", "Attendance >= 70", "Combined Rule"],
    "Complexity": [rule1_complexity, rule2_complexity, rule3_complexity],
    "Errors": [rule1_errors, rule2_errors, rule3_errors],
    "MDL_Score": [rule1_complexity + rule1_errors, rule2_complexity + rule2_errors, rule3_complexity +
})

display(mdl_results)
best_index = mdl_results["MDL_Score"].idxmin()
print(f"\nBest rule selected by MDL: {mdl_results.loc[best_index, 'Rule']}")
```

	Rule	Complexity	Errors	MDL_Score
0	Hours >= 6	3	1	4
1	Attendance >= 70	3	1	4
2	Combined Rule	7	1	8

Best rule selected by MDL: Hours >= 6

2. Bayes Optimal Classifier

We use weighted voting based on rule accuracy for a new student.

```
[6] accuracies = np.array([np.mean(p == actual) for p in [rule1_prediction, rule2_prediction, rule3_prediction]])
    weights = accuracies / accuracies.sum()

    new_student = {"Study_Hours": 6, "Attendance": 72, "Project_Done": 1}
    h1 = int(new_student["Study_Hours"] >= 6)
    h2 = int(new_student["Attendance"] >= 70)
    h3 = int(new_student["Study_Hours"] >= 5 and new_student["Attendance"] >= 65 and new_student["Project_Done"] == 1)

    hypothesis_predictions = np.array([h1, h2, h3])
    placed_prob = np.sum(weights * hypothesis_predictions)
    not_placed_prob = np.sum(weights * (1 - hypothesis_predictions))

    print(f"Placement probability: {placed_prob:.3f}")
    print(f"Bayes Optimal Prediction: {'Placed' if placed_prob > not_placed_prob else 'Not Placed'}")

... Placement probability: 1.000
    Bayes Optimal Prediction: Placed
```

3. Gibbs Sampling

Randomly selecting a rule based on its probability distribution.

```
np.random.seed(42)
rule_names = ["Study-hours rule", "Attendance rule", "Combined rule"]
selected_rule_index = np.random.choice([0, 1, 2], p=weights)

print(f"Selected rule: {rule_names[selected_rule_index]}")
print(f"Gibbs Prediction: {'Placed' if hypothesis_predictions[selected_rule_index] == 1 else 'Not Placed'}")

Selected rule: Attendance rule
Gibbs Prediction: Placed
```